
Internet Of Thinking II

Praktikum Pertemuan 4



Oleh:

NAMA : Bakti Ari Putra

NPM : 25782092

KELAS : TRI – 3D

Dosen:

Dr.Ir.Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027**

Kebutuhan Perangkat Keras

- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- LED 5mm (1 buah, atau 2 buah jika mengerjakan Tugas Mandiri)
- Resistor 220 Ohm (1 buah, atau 2 buah jika mengerjakan Tugas Mandiri)
- *Push Button / Tactile Switch* (1 buah)
- Resistor 10k Ohm (1 buah, untuk resistor *pull-down* tombol)
- Sensor Suhu & Kelembapan DHT11/22 (1 buah)
- Kabel *Jumper* (secukupnya)

Instalasi Pustaka Khusus (*Manual via GitHub*)

Karena berbasis teknologi Asinkron pihak ketiga, Anda harus mengunduh kode sumber pustaka ini secara manual dalam format .zip dari GitHub, lalu memasangnya ke Arduino IDE via menu **Sketch > Include Library > Add .ZIP Library...**

Silakan kunjungi tautan berikut dan unduh berkas ZIP dari menu Code > Download ZIP:

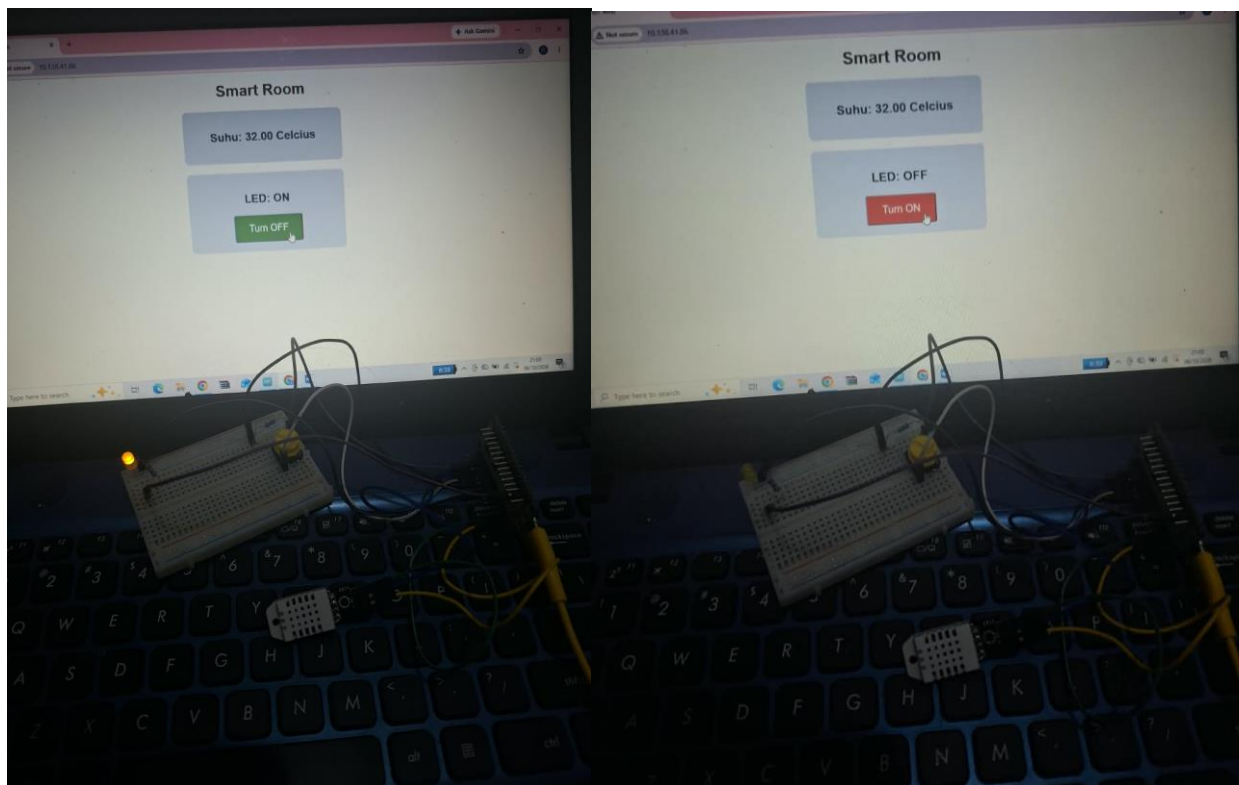
1. **ESPAsyncWebServer:** <https://github.com/ESP32Async/ESPAsyncWebServer>
2. **ESPAsyncTCP:** <https://github.com/ESP32Async/ESPAsyncTCP>

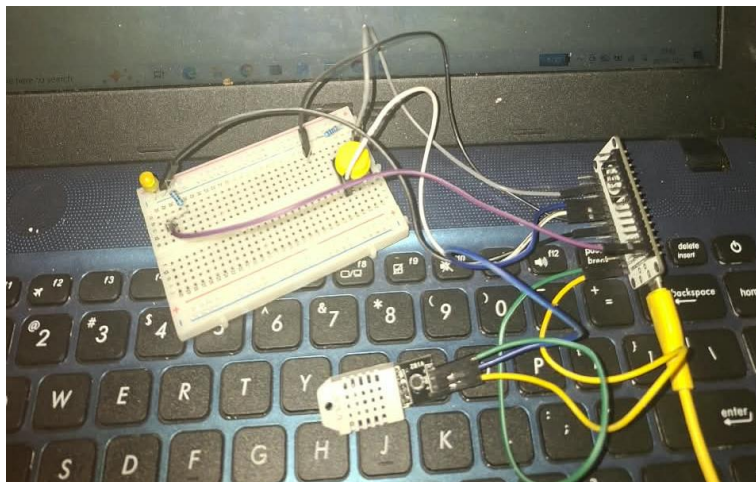
(Perhatian: Jika Anda menggunakan mikrokontroler seri ESP32 di luar praktikum ini, pustaka TCP-nya bernama *AsyncTCP*).

Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Langkah Kerja Rangkaian:

1. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin DATA / OUT DHT ke pin D4 (GPIO 2). (VCC DHT ke 3V3, GND ke GND).
3. Pasang Push Button pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur 3V3 (3.3V). Hubungkan kaki seberang tombol ke pin D2 (GPIO 4) NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki Resistor 10k Ohm, lalu hubungkan ujung kaki resistor lainnya ke jalur GND (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).





```

sketch_oct6a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESPAsyncTCP.h>
3  #include <ESPAsyncWebServer.h>
4  #include <DHT.h>
5
6  const char* ssid = "KING_Emyu";
7  const char* password = "baktiari";
8
9  // Konfigurasi Pin
10 const byte dhtPin = 2;    // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12;   // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19
Output  Serial Monitor X
Not connected. Select a board and a port to connect automatically.
IP Address: 10.138.41.86

```

Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

WebSocket lebih hemat bandwidth dibandingkan Meta Refresh HTTP karena WebSocket membuat satu koneksi yang tetap terbuka antara browser dan NodeMCU. Setelah koneksi terbentuk, NodeMCU dapat langsung mengirim data terbaru kepada browser ketika terjadi perubahan tanpa harus memuat ulang halaman.

Pada metode Meta Refresh HTTP, browser akan melakukan request HTTP baru secara berkala, misalnya setiap 5 detik. Walaupun tidak ada perubahan data, browser tetap meminta halaman dari NodeMCU. Hal tersebut menyebabkan penggunaan bandwidth dan proses komunikasi menjadi lebih banyak.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jika `delay(3000)` digunakan di dalam `void loop()`, maka NodeMCU akan **berhenti menjalankan proses selama 3 detik** setiap kali mencapai perintah tersebut. Hal ini berbahaya untuk sistem WebSocket karena selama `delay(3000)` berlangsung, NodeMCU tidak dapat merespons tombol fisik, memproses komunikasi WebSocket, maupun menjalankan tugas lain secara optimal.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

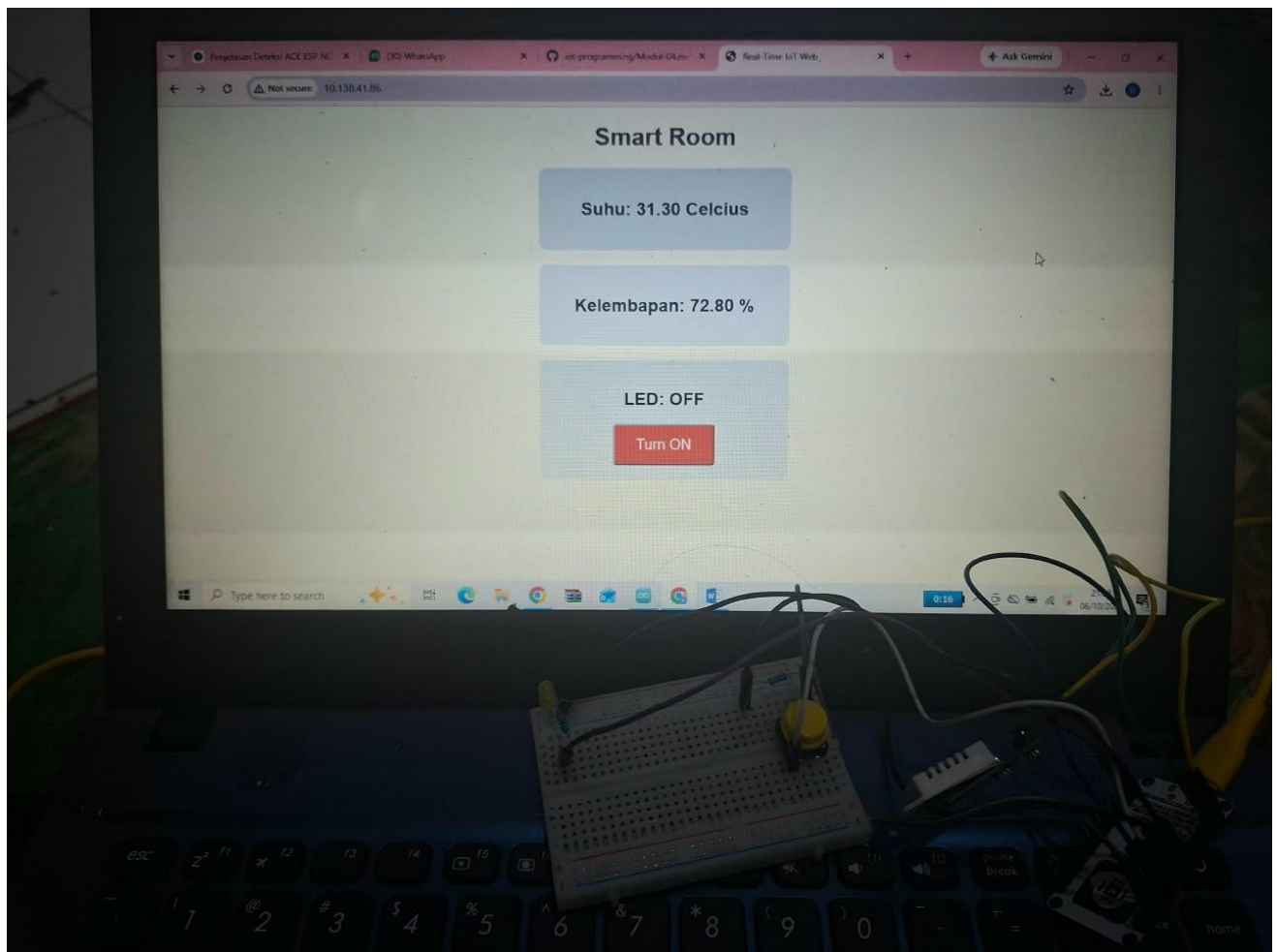
Pada fungsi JavaScript `onMessage()`, terdapat dua elemen `<span>` yang menjadi target injeksi atau pengubahan teks, yaitu `tempValue` dan `ledStatus`.

1. tempValue digunakan untuk menampilkan nilai suhu terbaru yang dikirim oleh NodeMCU. Pengubahannya dilakukan dengan document.getElementById('tempValue').innerHTML.
2. ledStatus digunakan untuk menampilkan status LED, yaitu ON atau OFF.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

1. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led": "...", "suhu": "...", "hum": "..."}).
2. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

```
sketch_oct6a.ino
12 // =====
13 // KONFIGURASI PIN
14 // =====
15 const byte dhtPin = 2; // D4 / GPIO 2
16 const byte buttonPin = 4; // D2 / GPIO 4
17 const byte ledPin = 12; // D6 / GPIO 12
18
19 // Sensor DHT22
20 DHT dht(dhtPin, DHT22);
21
22 // =====
23 // VARIABEL STATUS
24 // =====
25 bool ledState = false;
26
27 String currentTemp = "--";
28 String currentHum = "--";
29
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3') New Line 115200 baud
Client WebSocket #2 terhubung
Client WebSocket #1 terhubung
Client WebSocket #1 terhubung
r1010r0$00n11b|0000r0b00nn01nn0bb0p0$b0lr1p0n0010bn0n000b0nn'1001'000nn1'000nr000nb10p0n00r00b00bn0b00nn'1'000
WiFi Terhubung!
IP Address: 10.138.41.86
Web Server dimulai!
Client WebSocket #1 terhubung
Client WebSocket #2 terhubung
```



-Variabel penampung kelembapan:

```
String currentHum = "--";
```

-Membaca kelembapan:

```
float h = dht.readHumidity();
```

-Menyimpan nilai kelembapan:

```
if(!isnan(h)) {  
    currentHum = String(h, 2);  
}
```

-Menambahkan kelembapan ke JSON WebSocket:

```
jsonString += "\"hum\": \"" + currentHum + "\"}";
```

Kode lengkap fungsi `notifyClients()`:

```
1  ``cpp
2  String currentHum = "--";
3
4  void notifyClients() {
5      String jsonString = "{\"led\": \"" + String(ledState ? 1 : 0) + "\", ";
6      jsonString += "\"suhu\": \"" + currentTemp + "\", ";
7      jsonString += "\"hum\": \"" + currentHum + "\"}";
8      ws.textAll(jsonString);
9  }
10 ``
11
12 Pada timer `millis()` :
13
14 ``cpp
15 if ((millis() - lastTime) > 3000) {
16     float t = dht.readTemperature();
17     float h = dht.readHumidity();
18
19     if(!isnan(t)) {
20         currentTemp = String(t, 2);
21     }
22
23     if(!isnan(h)) {
24         currentHum = String(h, 2);
25     }
26
27     notifyClients();
28
29     lastTime = millis();
30 }
31 ``
32
```

Sisi

Front-End HTML

```
1  ``javascript
2  if(dataObj.hum !== undefined) {
3      document.getElementById('humValue').innerHTML = dataObj.hum;
4  }
5  ``
6
```

Sisi Front-End JavaScript

```
1  ``javascript
2  if(dataObj.hum !== undefined) {
3      document.getElementById('humValue').innerHTML = dataObj.hum;
4  }
5  ``
6
```